

▼ CS441: Applied ML - HW 2

▼ Part 1: Model Complexity and Tree-based Regressors

Include all the code for Part 1 in this section

```
import numpy as np
from google.colab import drive
%matplotlib inline
from matplotlib import pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso

# load data (modify to match your data directory or comment)
def load_temp_data():
    drive.mount('/content/drive')
    datadir = "/content/drive/My Drive/CS441/hw1/"
    T = np.load(datadir + 'temperature_data.npz')
    x_train, y_train, x_val, y_val, x_test, y_test, dates_train, dates_val, dates_test, feature_to_city, feature_to_day = \
    T['x_train'], T['y_train'], T['x_val'], T['y_val'], T['x_test'], T['y_test'], T['dates_train'], T['dates_val'], T['dates_test'],
    return (x_train, y_train, x_val, y_val, x_test, y_test, dates_train, dates_val, dates_test, feature_to_city, feature_to_day)

# plot one data point for listed cities and target temperature
def plot_temps(x, y, cities, feature_to_city, feature_to_day, target_date):
    nc = len(cities)
    ndays = 5
    xplot = np.array([-5,-4,-3,-2,-1])
    yplot = np.zeros((nc,ndays))
    for f in np.arange(len(x)):
        for c in np.arange(nc):
            if cities[c]==feature_to_city[f]:
                yplot[feature_to_day[f]+ndays,c] = x[f]
    plt.plot(xplot,yplot)
    plt.legend(cities)
    plt.plot(0, y, 'b*', markersize=10)
    plt.title('Predict Temp for Cleveland on ' + target_date)
    plt.xlabel('Day')
    plt.ylabel('Avg Temp (C)')
    plt.show()

# load data
(x_train, y_train, x_val, y_val, x_test, y_test, dates_train, dates_val, dates_test, feature_to_city, feature_to_day) = load_temp_data()

# to plot the errors
def plot_depth_error(max_depths, tree_train_err, tree_val_err, rf_train_err, rf_val_err, bt_train_err, bt_val_err):
    plt.figure(figsize=(15,15))
    plt.semilogx(max_depths, tree_train_err, 'r--',label='tree train')
    plt.semilogx(max_depths, tree_val_err, 'r.-', label='tree val')
    plt.semilogx(max_depths, rf_train_err, 'g--',label='RF train')
    plt.semilogx(max_depths, rf_val_err, 'g.-', label='RF val')
    plt.semilogx(max_depths, bt_train_err, 'b--',label='BT train')
    plt.semilogx(max_depths, bt_val_err, 'b.-', label='BT val')
    plt.ylabel('RMSE Error')
    plt.xlabel('Max Tree Depth')
    plt.xticks(max_depths, max_depths)
    plt.legend()
    plt.rcParams.update({'font.size': 20})
    plt.show()

from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor

max_depths = [2,4,8,16,32]

# usage examples
# DTR = DecisionTreeRegressor(random_state=0, max_depth=max_depths)
# RFR = RandomForestRegressor(random_state=0, max_depth=max_depths, max_features=1/3)
# GBR = GradientBoostingRegressor(random_state=0, max_depth=max_depths)
```

```
# Creating six arrays to hold the train and val RMSE values.
DTRTRMSE = []
RFRTRMSE = []
GBRTRMSE = []
DTRVRMSE = []
RFRVRMSE = []
GBRVRMSE = []

# Getting the train RMSE value for DecisionTreeRegressor when the max depth value is 2.
DTRMT1 = DecisionTreeRegressor(random_state=0, max_depth=2)
DTRMT1.fit(x_train, y_train)
DTRMT1P = DTRMT1.predict(x_train)
DTRTRMSE.append(np.sqrt(np.mean((DTRMT1P-y_train)**2)))

# Getting the val RMSE value for DecisionTreeRegressor when the max depth value is 4.
DTRMV1 = DecisionTreeRegressor(random_state=0, max_depth=2)
DTRMV1.fit(x_train, y_train)
DTRMV1P = DTRMV1.predict(x_val)
DTRVRMSE.append(np.sqrt(np.mean((DTRMV1P-y_val)**2)))

# Getting the train RMSE value for DecisionTreeRegressor when the max depth value is 4.
DTRMT2 = DecisionTreeRegressor(random_state=0, max_depth=4)
DTRMT2.fit(x_train, y_train)
DTRMT2P = DTRMT2.predict(x_train)
DTRTRMSE.append(np.sqrt(np.mean((DTRMT2P-y_train)**2)))

# Getting the val RMSE value for DecisionTreeRegressor when the max depth value is 4.
DTRMV2 = DecisionTreeRegressor(random_state=0, max_depth=4)
DTRMV2.fit(x_train, y_train)
DTRMV2P = DTRMV2.predict(x_val)
DTRVRMSE.append(np.sqrt(np.mean((DTRMV2P-y_val)**2)))

# Getting the train RMSE value for DecisionTreeRegressor when the max depth value is 8.
DTRMT3 = DecisionTreeRegressor(random_state=0, max_depth=8)
DTRMT3.fit(x_train, y_train)
DTRMT3P = DTRMT3.predict(x_train)
DTRTRMSE.append(np.sqrt(np.mean((DTRMT3P-y_train)**2)))

# Getting the val RMSE value for DecisionTreeRegressor when the max depth value is 8.
DTRMV3 = DecisionTreeRegressor(random_state=0, max_depth=8)
DTRMV3.fit(x_train, y_train)
DTRMV3P = DTRMV3.predict(x_val)
DTRVRMSE.append(np.sqrt(np.mean((DTRMV3P-y_val)**2)))

# Getting the train RMSE value for DecisionTreeRegressor when the max depth value is 16.
DTRMT4 = DecisionTreeRegressor(random_state=0, max_depth=16)
DTRMT4.fit(x_train, y_train)
DTRMT4P = DTRMT4.predict(x_train)
DTRTRMSE.append(np.sqrt(np.mean((DTRMT4P-y_train)**2)))

# Getting the val RMSE value for DecisionTreeRegressor when the max depth value is 16.
DTRMV4 = DecisionTreeRegressor(random_state=0, max_depth=16)
DTRMV4.fit(x_train, y_train)
DTRMV4P = DTRMT4.predict(x_val)
DTRVRMSE.append(np.sqrt(np.mean((DTRMV4P-y_val)**2)))

# Getting the train RMSE value for DecisionTreeRegressor when the max depth value is 32.
DTRMT5 = DecisionTreeRegressor(random_state=0, max_depth=32)
DTRMT5.fit(x_train, y_train)
DTRMT5P = DTRMT5.predict(x_train)
DTRTRMSE.append(np.sqrt(np.mean((DTRMT5P-y_train)**2)))

# Getting the val RMSE value for DecisionTreeRegressor when the max depth value is 32.
DTRMV5 = DecisionTreeRegressor(random_state=0, max_depth=32)
DTRMV5.fit(x_train, y_train)
DTRMV5P = DTRMV5.predict(x_val)
DTRVRMSE.append(np.sqrt(np.mean((DTRMV5P-y_val)**2)))
```

```
# Getting the train RMSE value for RandomForestRegressor when the max depth value is 2.
RFRMT1 = RandomForestRegressor(random_state=0, max_depth=2, max_features=1/3)
RFRMT1.fit(x_train, y_train)
RFRMT1P = RFRMT1.predict(x_train)
RFRTRMSE.append(np.sqrt(np.mean((RFRMT1P-y_train)**2)))

# Getting the train RMSE value for RandomForestRegressor when the max depth value is 4.
RFRMT2 = RandomForestRegressor(random_state=0, max_depth=4, max_features=1/3)
RFRMT2.fit(x_train, y_train)
RFRMT2P = RFRMT2.predict(x_train)
RFRTRMSE.append(np.sqrt(np.mean((RFRMT2P-y_train)**2)))

# Getting the train RMSE value for RandomForestRegressor when the max depth value is 8.
RFRMT3 = RandomForestRegressor(random_state=0, max_depth=8, max_features=1/3)
RFRMT3.fit(x_train, y_train)
RFRMT3P = RFRMT3.predict(x_train)
RFRTRMSE.append(np.sqrt(np.mean((RFRMT3P-y_train)**2)))

# Getting the train RMSE value for RandomForestRegressor when the max depth value is 16.
RFRMT4 = RandomForestRegressor(random_state=0, max_depth=16, max_features=1/3)
RFRMT4.fit(x_train, y_train)
RFRMT4P = RFRMT4.predict(x_train)
RFRTRMSE.append(np.sqrt(np.mean((RFRMT4P-y_train)**2)))

# Getting the train RMSE value for RandomForestRegressor when the max depth value is 32.
RFRMT5 = RandomForestRegressor(random_state=0, max_depth=32, max_features=1/3)
RFRMT5.fit(x_train, y_train)
RFRMT5P = RFRMT5.predict(x_train)
RFRTRMSE.append(np.sqrt(np.mean((RFRMT5P-y_train)**2)))

# Getting the val RMSE value for RandomForestRegressor when the max depth value is 2.
RFRMV1 = RandomForestRegressor(random_state=0, max_depth=2, max_features=1/3)
RFRMV1.fit(x_train, y_train)
RFRMV1P = RFRMT1.predict(x_val)
RFRVRMSE.append(np.sqrt(np.mean((RFRMV1P-y_val)**2)))

# Getting the val RMSE value for RandomForestRegressor when the max depth value is 4.
RFRMV2 = RandomForestRegressor(random_state=0, max_depth=4, max_features=1/3)
RFRMV2.fit(x_train, y_train)
RFRMV2P = RFRMT2.predict(x_val)
RFRVRMSE.append(np.sqrt(np.mean((RFRMV2P-y_val)**2)))

# Getting the val RMSE value for RandomForestRegressor when the max depth value is 8.
RFRMV3 = RandomForestRegressor(random_state=0, max_depth=8, max_features=1/3)
RFRMV3.fit(x_train, y_train)
RFRMV3P = RFRMT3.predict(x_val)
RFRVRMSE.append(np.sqrt(np.mean((RFRMV3P-y_val)**2)))

# Getting the val RMSE value for RandomForestRegressor when the max depth value is 16.
RFRMV4 = RandomForestRegressor(random_state=0, max_depth=16, max_features=1/3)
RFRMV4.fit(x_train, y_train)
RFRMV4P = RFRMT4.predict(x_val)
RFRVRMSE.append(np.sqrt(np.mean((RFRMV4P-y_val)**2)))

# Getting the val RMSE value for RandomForestRegressor when the max depth value is 32.
RFRMV5 = RandomForestRegressor(random_state=0, max_depth=32, max_features=1/3)
RFRMV5.fit(x_train, y_train)
RFRMV5P = RFRMT5.predict(x_val)
RFRVRMSE.append(np.sqrt(np.mean((RFRMV5P-y_val)**2)))

# Getting the train RMSE value for GradientBoostingRegressor when the max depth value is 2.
GBRMT1 = GradientBoostingRegressor(random_state=0, max_depth=2)
GBRMT1.fit(x_train, y_train)
GBRMT1P = GBRMT1.predict(x_train)
GBRTRMSE.append(np.sqrt(np.mean((GBRMT1P-y_train)**2)))

# Getting the train RMSE value for GradientBoostingRegressor when the max depth value is 4.
GBRMT2 = GradientBoostingRegressor(random_state=0, max_depth=4)
GBRMT2.fit(x_train, y_train)
```

```
GBRMT2P = GBRMT2.predict(x_train)
GBRTRMSE.append(np.sqrt(np.mean((GBRMT2P-y_train)**2)))

# Getting the train RMSE value for GradientBoostingRegressor when the max depth value is 8.
GBRMT3 = GradientBoostingRegressor(random_state=0, max_depth=8)
GBRMT3.fit(x_train, y_train)
GBRMT3P = GBRMT3.predict(x_train)
GBRTRMSE.append(np.sqrt(np.mean((GBRMT3P-y_train)**2)))

# Getting the train RMSE value for GradientBoostingRegressor when the max depth value is 16.
GBRMT4 = GradientBoostingRegressor(random_state=0, max_depth=16)
GBRMT4.fit(x_train, y_train)
GBRMT4P = GBRMT4.predict(x_train)
GBRTRMSE.append(np.sqrt(np.mean((GBRMT4P-y_train)**2)))

# Getting the train RMSE value for GradientBoostingRegressor when the max depth value is 32.
GBRMT5 = GradientBoostingRegressor(random_state=0, max_depth=32)
GBRMT5.fit(x_train, y_train)
GBRMT5P = GBRMT5.predict(x_train)
GBRTRMSE.append(np.sqrt(np.mean((GBRMT5P-y_train)**2)))

# Getting the val RMSE value for GradientBoostingRegressor when the max depth value is 2.
GBRMV1 = GradientBoostingRegressor(random_state=0, max_depth=2)
GBRMV1.fit(x_train, y_train)
GBRMV1P = GBRMV1.predict(x_val)
GBRVRMSE.append(np.sqrt(np.mean((GBRMV1P-y_val)**2)))

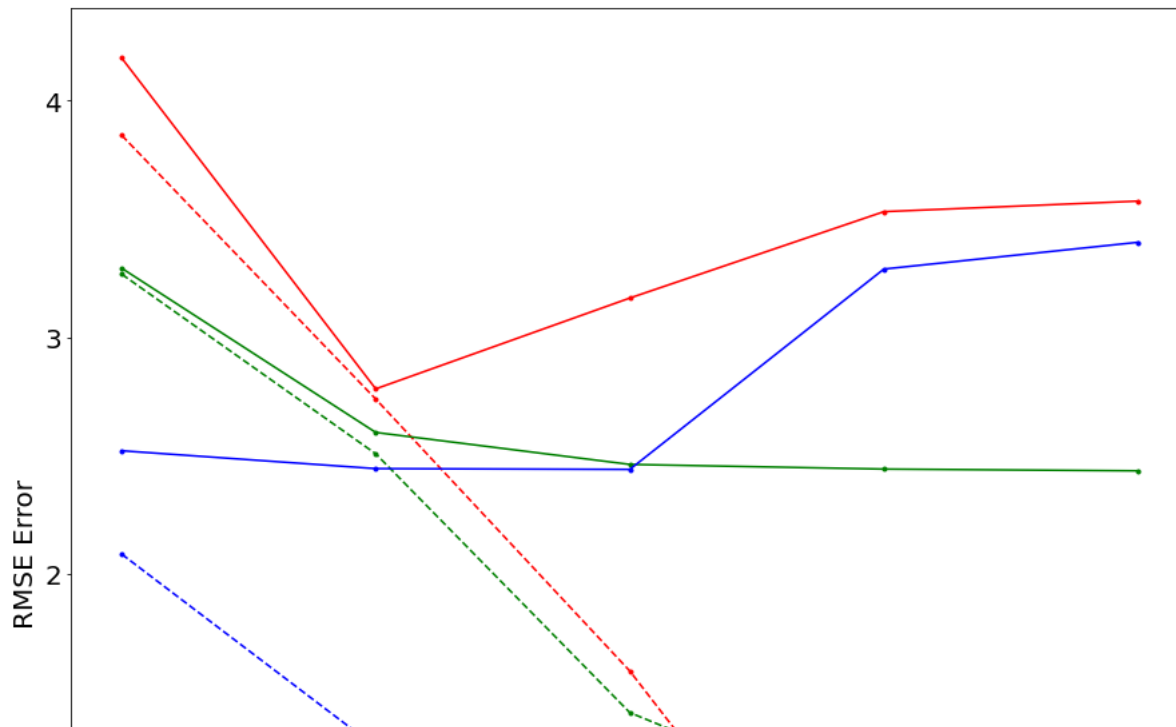
# Getting the val RMSE value for GradientBoostingRegressor when the max depth value is 4.
GBRMV2 = GradientBoostingRegressor(random_state=0, max_depth=4)
GBRMV2.fit(x_train, y_train)
GBRMV2P = GBRMV2.predict(x_val)
GBRVRMSE.append(np.sqrt(np.mean((GBRMV2P-y_val)**2)))

# Getting the val RMSE value for GradientBoostingRegressor when the max depth value is 8.
GBRMV3 = GradientBoostingRegressor(random_state=0, max_depth=8)
GBRMV3.fit(x_train, y_train)
GBRMV3P = GBRMV3.predict(x_val)
GBRVRMSE.append(np.sqrt(np.mean((GBRMV3P-y_val)**2)))

# Getting the val RMSE value for GradientBoostingRegressor when the max depth value is 16.
GBRMV4 = GradientBoostingRegressor(random_state=0, max_depth=16)
GBRMV4.fit(x_train, y_train)
GBRMV4P = GBRMV4.predict(x_val)
GBRVRMSE.append(np.sqrt(np.mean((GBRMV4P-y_val)**2)))

# Getting the val RMSE value for GradientBoostingRegressor when the max depth value is 32.
GBRMV5 = GradientBoostingRegressor(random_state=0, max_depth=32)
GBRMV5.fit(x_train, y_train)
GBRMV5P = GBRMV5.predict(x_val)
GBRVRMSE.append(np.sqrt(np.mean((GBRMV5P-y_val)**2)))

# Plotting the depth error chart by calling the plot_depth_error function.
PDE = plot_depth_error(max_depths, DTRTRMSE, DTRVRMSE, RFRTRMSE, RFRVRMSE, GBRTRMSE, GBRVRMSE)
```



▼ Part 2: MLPs with MNIST

Include all the code for Part 2 in this section

```
# initialization code
import numpy as np
from keras.datasets import mnist
%matplotlib inline
from matplotlib import pyplot as plt
from scipy import stats
import torch
import torch.nn as nn

# loads MNIST data and reformat to 768-d vectors with values in range 0 to 1
# splits into train/val/test sets and provides indices for subsets of train
def load_mnist():
    (x_train, y_train), (x_test, y_test) = mnist.load_data()
    x_train = np.reshape(x_train, (len(x_train), 28*28))
    x_test = np.reshape(x_test, (len(x_test), 28*28))
    maxval = x_train.max()
    x_train = x_train/maxval
    x_test = x_test/maxval
    x_val = x_train[:10000]
    y_val = y_train[:10000]
    x_train = x_train[10000:]
    y_train = y_train[10000:]
    train_indices = dict()
    train_indices['xs'] = np.arange(50)
    train_indices['s'] = np.arange(500)
    train_indices['m'] = np.arange(5000)
    train_indices['all'] = np.arange(50000)
    return (x_train, y_train), (x_val, y_val), (x_test, y_test), train_indices

# displays a set of mnist images
def display_mnist(x, subplot_rows=1, subplot_cols=1):
    if subplot_rows>1 or subplot_cols>1:
        fig, ax = plt.subplots(subplot_rows, subplot_cols, figsize=(15,15))
        for i in np.arange(len(x)):
            ax[i].imshow(np.reshape(x[i], (28,28)), cmap='gray')
            ax[i].axis('off')
    else:
        plt.imshow(np.reshape(x, (28,28)), cmap='gray')
        plt.axis('off')
    plt.show()

# counts the number of examples per class
```

```

def class_count_mnist(y):
    count = np.zeros((10,), dtype='uint32')
    for i in np.arange(10):
        count[i] = sum(y==i)
    return count

(x_train, y_train), (x_val, y_val), (x_test, y_test), train_indices = load_mnist()

# Sets device to "cuda" if a GPU is available (in Colabs, enable GPU by Edit->Notebook Settings-->Hardware Accelerator=GPU)
device = "cuda" if torch.cuda.is_available() else 'cpu'

def display_error_curves(num_epochs, training_losses, validation_losses):
    """
    Plots the training and validation loss curves
    """

    plt.plot(range(num_epochs), training_losses, label="Training Loss")
    plt.plot(range(num_epochs), validation_losses, label="Validation Loss")

    # Add in a title and axes labels
    plt.title('Training and Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')

    # Display the plot
    plt.legend(loc='best')
    plt.show()

# Creating a training set.
trainset = torch.utils.data.TensorDataset(torch.Tensor(x_train), torch.Tensor(np.eye(10)[y_train]))

# Creating a training loader.
train_loader = torch.utils.data.DataLoader(trainset, batch_size=256, shuffle=True, num_workers=1)

# Creating a validation set.
valset = torch.utils.data.TensorDataset(torch.Tensor(x_val), torch.Tensor(np.eye(10)[y_val]))

# Creating a validation loader.
val_loader = torch.utils.data.DataLoader(valset, batch_size=256, shuffle=False, num_workers=1)

# Defining a model called MLP.
class MLP(nn.Module):
    # Creating a function called __init__.
    def __init__(self, input_size, hidden_size, output_size):
        super(MLP, self).__init__()

        # Creating variables called linearOne, reluOne, and linearTwo..
        self.linearOne = nn.Linear(input_size, hidden_size)

        self.reluOne = nn.ReLU()

        self.linearTwo = nn.Linear(hidden_size, output_size)

    # Creating a function called forward.
    def forward(self, x1):

        # Setting x2 equal to linearOne, reluOne, and linearTwo.
        x2 = self.linearOne(x1)
        x2 = self.reluOne(x2)
        x2 = self.linearTwo(x2)

        return x2 # Returning x2.

# Creating a function called train_MLP_mnist.
def train_MLP_mnist(train_loader, val_loader, lr=0.12, num_epochs=100):
    """
    Train a MLP
    Input: X[i] is the ith training data. y[i] is the ith training label. alpha is the count prior
    Output: return a trained MLP
    """
    # Declaring input_size, hidden_size, and output_size.
    input_size = 784
    hidden_size = 64
    output_size = 10

```

```

# Instantiating the model.
mlp = MLP(input_size, hidden_size, output_size).to(device)

# Setting optimizer and loss function
optimizer = torch.optim.SGD(mlp.parameters(), lr)
loss_function = nn.CrossEntropyLoss()

# Creating the train_loss and val_loss arrays.
train_loss = []
val_loss = []

# Training the model.
for epoch in range(0, num_epochs):

    # Creating the t_loss and v_loss variables.
    t_loss = 0.0
    v_loss = 0.0

    # Iterating over the DataLoader for training data.
    for i, data in enumerate(train_loader, 0):

        inputs, targets = data # Getting inputs.

        inputs = inputs.to(device) # Sending inputs to device.

        targets = targets.to(device) # Sending targets to device.

        optimizer.zero_grad() # Zeroing the gradients.

        outputs = mlp(inputs) # Computing logit scores for current batch.

        loss = loss_function(outputs, targets) # Computing loss.

        loss.backward() # Here is the backprop loss.

        optimizer.step() # Updating weights.

        t_loss += loss.item() # Adding loss to t_loss.

    # Iterating over the DataLoader for validation data.
    for i, data in enumerate(val_loader, 0):

        inputs, targets = data # Get inputs

        inputs = inputs.to(device) # Sending inputs to device.

        targets = targets.to(device) # Sending targets to device.

        outputs = mlp(inputs) # Computing logit scores for current batch.

        loss = loss_function(outputs, targets) # Computing loss.

        v_loss += loss.item() # Adding loss to v_loss.

    train_loss.append(t_loss/i) # Appending to the train_loss array.

    val_loss.append(v_loss/i) # Appending to the val_loss array.

# Printing the loss information for the training loader.
print("The train array is: ", train_loss)
print("The 25 epoch is: ", train_loss[24])
print("The 50 epoch is: ", train_loss[49])
print("The 100 epoch is: ", train_loss[99])

# Printing the loss information for the validation loader.
print("The val array is: ", val_loss)
print("The 25 epoch is: ", val_loss[24])
print("The 50 epoch is: ", val_loss[49])
print("The 100 epoch is: ", val_loss[99])

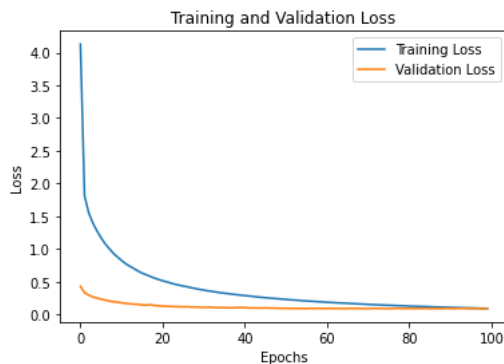
# Here the loss curves are displayed.
display_error_curves(num_epochs, train_loss, val_loss)

# Here mlp is returned.
return mlp

```

```
# Here is the information when the learning rate is improved to 0.12.
train_MLP_mnist(train_loader, val_loader)
```

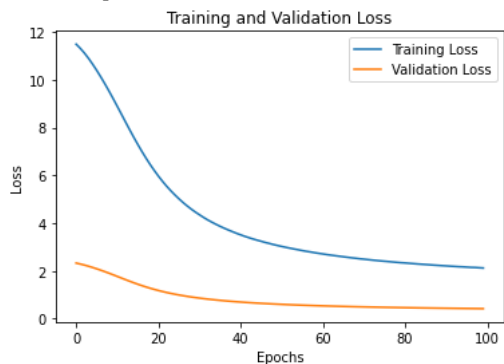
```
The train array is: [4.128838999913289, 1.8139234150831516, 1.5544168253739674, 1.3997352975301254, 1.2772936450365262, 1.17
The 25 epoch is: 0.4462321714904064
The 50 epoch is: 0.2335760754127151
The 100 epoch is: 0.09166320979308623
The val array is: [0.4283412561202661, 0.33243945088142, 0.29522515680545414, 0.26789485147366154, 0.2492739703410711, 0.23
The 25 epoch is: 0.11770099458786157
The 50 epoch is: 0.09133672647369213
The 100 epoch is: 0.08955111010716511
```



```
MLP(
  (fc1): Linear(in_features=784, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=10, bias=True)
)
```

```
# Here is the information when the learning rate is 0.001.
train_MLP_mnist(train_loader, val_loader)
```

```
The train array is: [11.493755695147392, 11.32093959588271, 11.123846145776602, 10.902184437482784, 10.657751456285135, 10.
The 25 epoch is: 5.1837843121626435
The 50 epoch is: 3.063143742390168
The 100 epoch is: 2.1216772359151106
The val array is: [2.327859248870458, 2.289075900346805, 2.245546738306681, 2.1970420556190686, 2.1444911100925546, 2.08855
The 25 epoch is: 1.0256019372206469
The 50 epoch is: 0.6079619289972843
The 100 epoch is: 0.4201350739369026
```



```
MLP(
  (fc1): Linear(in_features=784, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=10, bias=True)
)
```

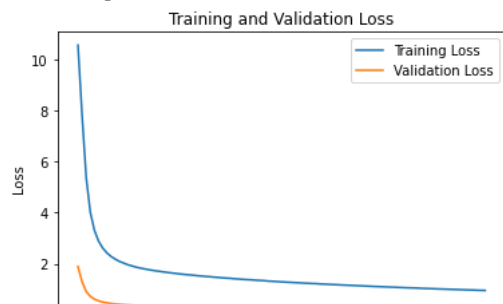
```
# Here is the information when the learning rate is 0.01.
train_MLP_mnist(train_loader, val_loader)
```



```

The train array is: [10.548699513459818, 7.824162107247573, 5.374181501376323, 4.0182264890426245, 3.3042000379317846, 2.886
The 25 epoch is: 1.6127056941772118
The 50 epoch is: 1.2934683106648617
The 100 epoch is: 0.9522466063499451
The val array is: [1.8881280391644208, 1.2919098261075141, 0.9127717293225802, 0.7198638518651327, 0.6126800125990158, 0.546
The 25 epoch is: 0.31828030714621913
The 50 epoch is: 0.2607436986305775
The 100 epoch is: 0.20089576947383392

```



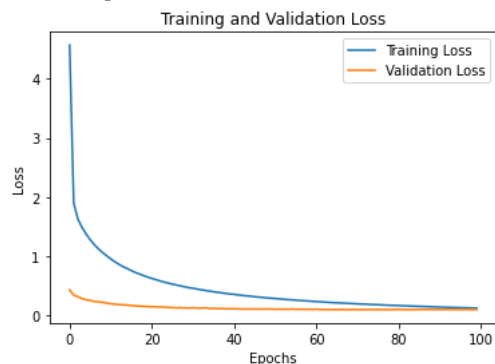
Here is the information when the learning rate is 0.1.

```
train_MLP_mnist(train_loader, val_loader)
```

```

The train array is: [4.569005897411933, 1.8987049811925643, 1.6298479369053473, 1.48836133762812, 1.3757569446013524, 1.2785
The 25 epoch is: 0.5433568798769743
The 50 epoch is: 0.2882781217877681
The 100 epoch is: 0.11961032705715834
The val array is: [0.4294982835268363, 0.340291393108857, 0.31452204096011627, 0.28258971258615834, 0.26305919618178636, 0.2
The 25 epoch is: 0.13234645261978492
The 50 epoch is: 0.10586874272960883
The 100 epoch is: 0.09638118820312695

```



```

MLP(
  (fc1): Linear(in_features=784, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=10, bias=True)
)

```

Here is the information when the learning rate is 1.

```
train_MLP_mnist(train_loader, val_loader)
```

```

The train array is: [2.5617285076624308, 0.9814620428742507, 0.7452133658986825, 0.6236843288135834, 0.5289659693073003, 0.4
The 25 epoch is: 0.09593827351450156
The 50 epoch is: 0.0129894006322735
The 100 epoch is: 0.0036619203683818714
The val array is: [0.2744575486733363, 0.17472110168100932, 0.17648307845378533, 0.14755095106860003, 0.13991997617846116, (
The 25 epoch is: 0.12205856621193771
The 50 epoch is: 0.1434255401359107
The 100 epoch is: 0.16689701690726602

```

```

# experiments code
print(device) # make sure you're using GPU instance
torch.manual_seed(0) # to avoid randomness, but if you wanted to create an ensemble, you should not use a manual seed

```

```

cuda
<torch._C.Generator at 0x7efdc9f9f5b0>
% | |

```

▼ Part 3: Predicting Penguin Species

Include all your code for part 3 in this section.

```

%%
import numpy as np
from google.colab import drive
%matplotlib inline
from matplotlib import pyplot as plt
import pandas as pd
import seaborn as sns
#styling preferences for sns
sns.set_style('whitegrid')
sns.set_context('poster')
drive.mount('/content/gdrive/')
datadir = "/content/gdrive/MyDrive/CS441/hw2/"
df_penguins = pd.read_csv(datadir + 'penguins_size.csv')
df_penguins.head(10)

def get_penguin_xy(df_penguins):
    data = np.array(df_penguins[['island', 'culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']])
    y = df_penguins['species']
    ui = np.unique(data[:,0])
    us = np.unique(data[:,1])
    X = np.zeros((len(y), 10))
    for i in range(len(y)):
        f = 0
        for j in range(len(ui)):
            if data[i, f]==ui[j]:
                X[i, f+j] = 1
        X[i, 3:7] = data[i, 1:5]
        for j in range(len(us)):
            if data[i, 5]==us[j]:
                X[i, f+j] = 1
    feature_names = ['island_biscoe', 'island_dream', 'island_torgersen', 'culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm']
    X = pd.DataFrame(X, columns=feature_names)
    return(X, y, feature_names, np.unique(y))

Mounted at /content/gdrive/

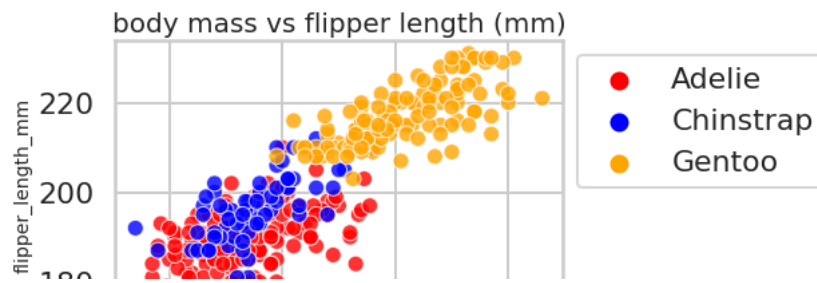
palette = ["red", "blue", "orange"]

# https://seaborn.pydata.org/generated/seaborn.scatterplot.html
sns.scatterplot(data=df_penguins, x = 'body_mass_g', y = 'flipper_length_mm',
                hue = 'species', palette=palette, alpha=0.8)

plt.xlabel('body mass', fontsize=14)
plt.ylabel('flipper_length_mm', fontsize=14)
plt.title('body mass vs flipper length (mm)', fontsize=20)
plt.legend(bbox_to_anchor=(1.0, 1.0), loc='upper left')
plt.show()

# create other visualizations

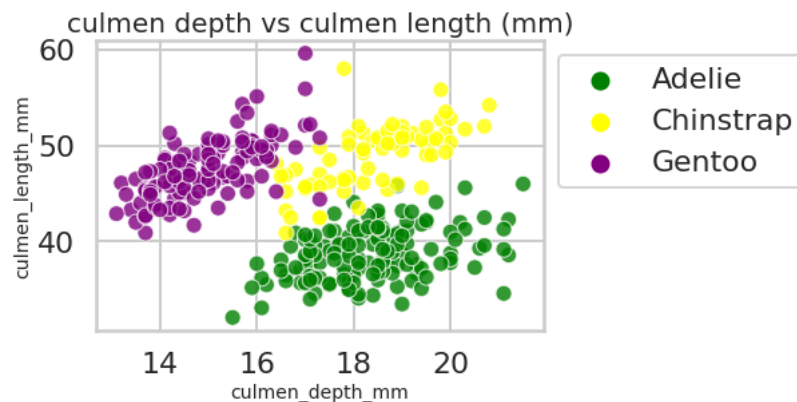
```



```
# Creating a palette with three different colors.
palette = ["green", "yellow", "purple"]

# Letting the scatterplot know about the features that it will be using.
sns.scatterplot(data=df_penguins, x = 'culmen_depth_mm', y = 'culmen_length_mm',
                hue = 'species', palette=palette, alpha=0.8)

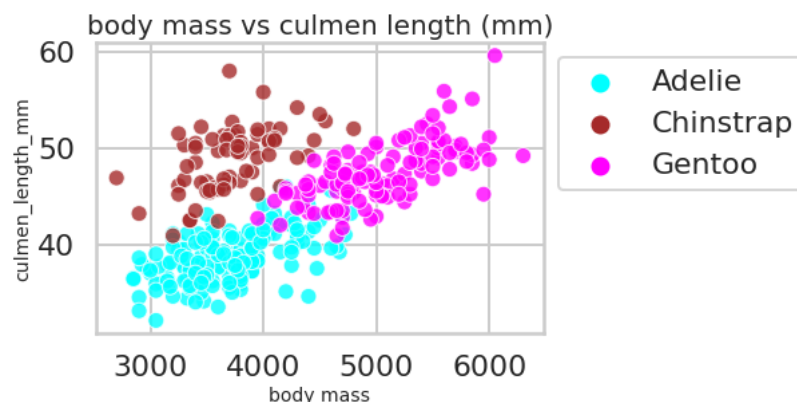
# Plotting the actual scatterplot.
plt.xlabel('culmen_depth_mm', fontsize=14)
plt.ylabel('culmen_length_mm', fontsize=14)
plt.title('culmen depth vs culmen length (mm)', fontsize=20)
plt.legend(bbox_to_anchor=(1.0, 1.0), loc='upper left')
plt.show()
```



```
# Creating a palette with three different colors.
palette = ["cyan", "brown", "magenta"]

# Letting the scatterplot know about the features that it will be using.
sns.scatterplot(data=df_penguins, x = 'body_mass_g', y = 'culmen_length_mm',
                hue = 'species', palette=palette, alpha=0.8)

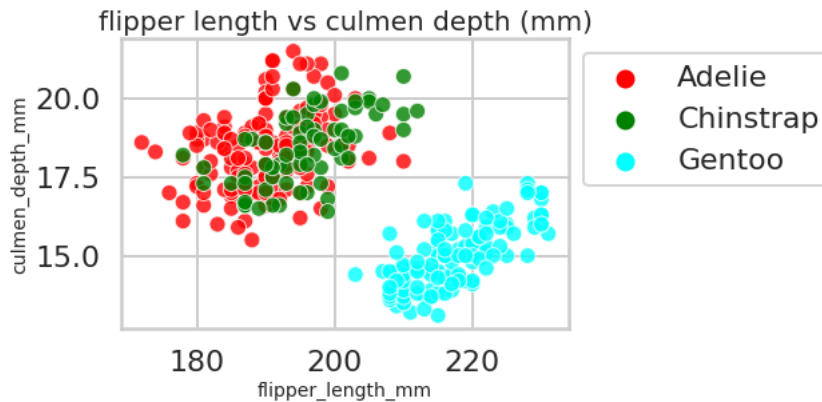
# Plotting the actual scatterplot.
plt.xlabel('body mass', fontsize=14)
plt.ylabel('culmen_length_mm', fontsize=14)
plt.title('body mass vs culmen length (mm)', fontsize=20)
plt.legend(bbox_to_anchor=(1.0, 1.0), loc='upper left')
plt.show()
```



```
# Creating a palette with three different colors.
palette = ["red", "green", "cyan"]
```

```
# Letting the scatterplot know about the features that it will be using.
sns.scatterplot(data=df_penguins, x = 'flipper_length_mm', y = 'culmen_depth_mm',
               hue = 'species', palette=palette, alpha=0.8)

# Plotting the actual scatterplot.
plt.xlabel('flipper_length_mm', fontsize=14)
plt.ylabel('culmen_depth_mm', fontsize=14)
plt.title('flipper length vs culmen depth (mm)', fontsize=20)
plt.legend(bbox_to_anchor=(1.0, 1.0), loc='upper left')
plt.show()
```



```
# Importing DecisionTreeClassifier, and plot_tree from sklearn.tree.
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

# Calling the get_penguin_xy function.
X, y, feature_names, class_names = get_penguin_xy(df_penguins)

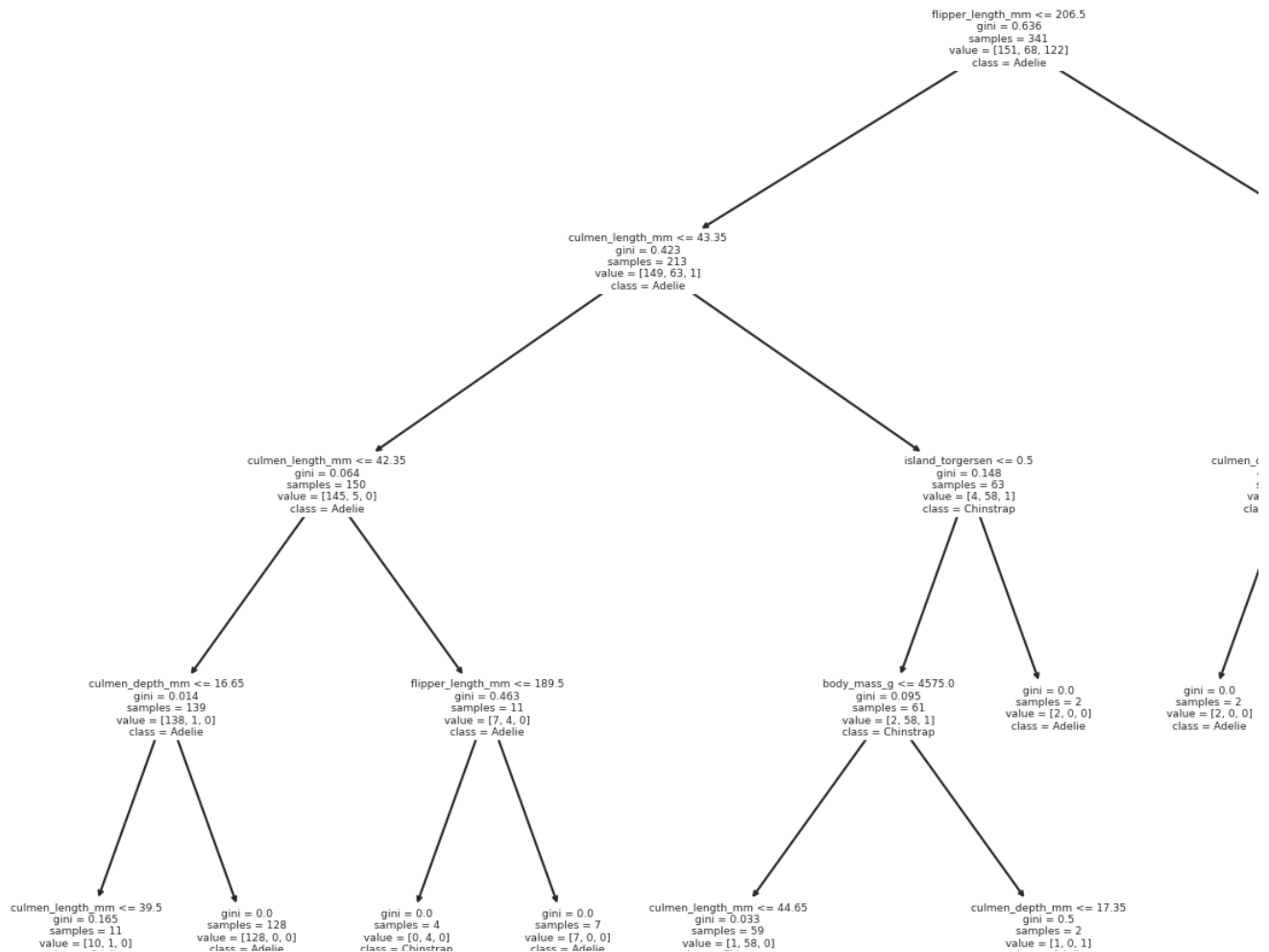
# Calling the Decision Tree Classifier model with the criterion set to gini.
DTC = DecisionTreeClassifier(criterion='gini')

# Fitting the Decision Tree Classifier model.
DTC.fit(X, y)

# Plotting the figure with a figure size of 25 X 25.
plt.figure(figsize=(25, 25))

# Calling the plot tree with the model, feature_names, and class_names passed to it.
plot_tree(DTC, feature_names=feature_names, class_names=class_names)

# Showing the plot that was created.
plt.show()
```



```
# design a classification model
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
```

```
X, y, feature_names, class_names = get_penguin_xy(df_penguins)
```

```
# Using the logistic regression model to calculate the cross validation score accuracy.
model = LogisticRegression(solver='lbfgs', max_iter=10000, C=4.0)
```

```
scores = cross_val_score(model, np.array(X), np.array(y), cv=5)
print('CV Accuracy: {}'.format(scores.mean()))
```

```
CV Accuracy: 0.9970588235294118
```

Part 3: Stretch Goals

Include any new code needed for Part 3 here

```
# Creating a function called train_MLP_mnist.
def train_MLP_mnist(train_loader, val_loader, lr=0.1, num_epochs=100):
    ...

    Train a MLP
    Input: X[i] is the ith training data. y[i] is the ith training label. alpha is the count prior
    Output: return a trained MLP
    ...

    # Declaring input_size, hidden_size, and output_size.
    input_size = 784
    hidden_size = 128
    output_size = 10

    # Instantiating the model.
    mlp = MLP(input_size, hidden_size, output_size).to(device)
```

```

# Setting optimizer and loss function
optimizer = torch.optim.SGD(mlp.parameters(), lr)
loss_function = nn.CrossEntropyLoss()

# Creating the train_loss and val_loss arrays.
train_loss = []
val_loss = []

# Training the model.
for epoch in range(0, num_epochs):

    # Creating the t_loss and v_loss variables.
    t_loss = 0.0
    v_loss = 0.0

    # Iterating over the DataLoader for training data.
    for i, data in enumerate(train_loader, 0):

        inputs, targets = data # Getting inputs.

        inputs = inputs.to(device) # Sending inputs to device.

        targets = targets.to(device) # Sending targets to device.

        optimizer.zero_grad() # Zeroing the gradients.

        outputs = mlp(inputs) # Computing logit scores for current batch.

        loss = loss_function(outputs, targets) # Computing loss.

        loss.backward() # Here is the backprop loss.

        optimizer.step() # Updating weights.

        t_loss += loss.item() # Adding loss to t_loss.

    # Iterating over the DataLoader for validation data.
    for i, data in enumerate(val_loader, 0):

        inputs, targets = data # Get inputs

        inputs = inputs.to(device) # Sending inputs to device.

        targets = targets.to(device) # Sending targets to device.

        outputs = mlp(inputs) # Computing logit scores for current batch.

        loss = loss_function(outputs, targets) # Computing loss.

        v_loss += loss.item() # Adding loss to v_loss.

    train_loss.append(t_loss/i) # Appending to the train_loss array.

    val_loss.append(v_loss/i) # Appending to the val_loss array.

# Printing the loss information for the training loader.
print("The train array is: ", train_loss)
print("The 25 epoch is: ", train_loss[24])
print("The 50 epoch is: ", train_loss[49])
print("The 100 epoch is: ", train_loss[99])

# Printing the loss information for the validation loader.
print("The val array is: ", val_loss)
print("The 25 epoch is: ", val_loss[24])
print("The 50 epoch is: ", val_loss[49])
print("The 100 epoch is: ", val_loss[99])

# Here the loss curves are displayed.
display_error_curves(num_epochs, train_loss, val_loss)

# Here mlp is returned.
return mlp

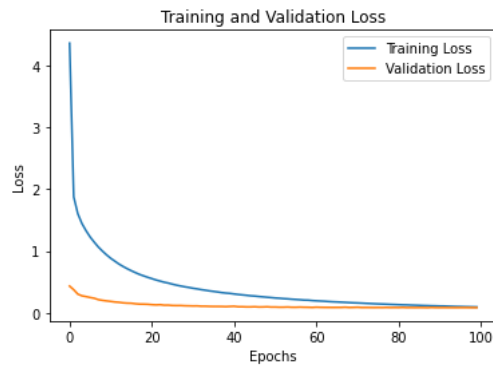
# Here is the information when the hidden layer size is improved to 128.
train_MLP_mnist(train_loader, val_loader)

```

```

The train array is: [4.364155830481113, 1.8772769558888216, 1.6030327994854023, 1.4449372108166034, 1.3285682805073566, 1.24
The 25 epoch is: 0.48164058085053396
The 50 epoch is: 0.24676089896223483
The 100 epoch is: 0.09492775738143767
The val array is: [0.4321668870173968, 0.373566522812232, 0.30263798091656124, 0.2752164124678343, 0.2626240012737421, 0.246
The 25 epoch is: 0.12423976233754402
The 50 epoch is: 0.0932714535544316
The 100 epoch is: 0.08410414064732882

```



```

MLP(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

```

```

# Importing DecisionTreeClassifier, and plot_tree from sklearn.tree.
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

# Calling the get_penguin_xy function.
X, y, feature_names, class_names = get_penguin_xy(df_penguins)

# Calling the Decision Tree Classifier model with the criterion set to gini.
DTC = DecisionTreeClassifier(criterion='gini')

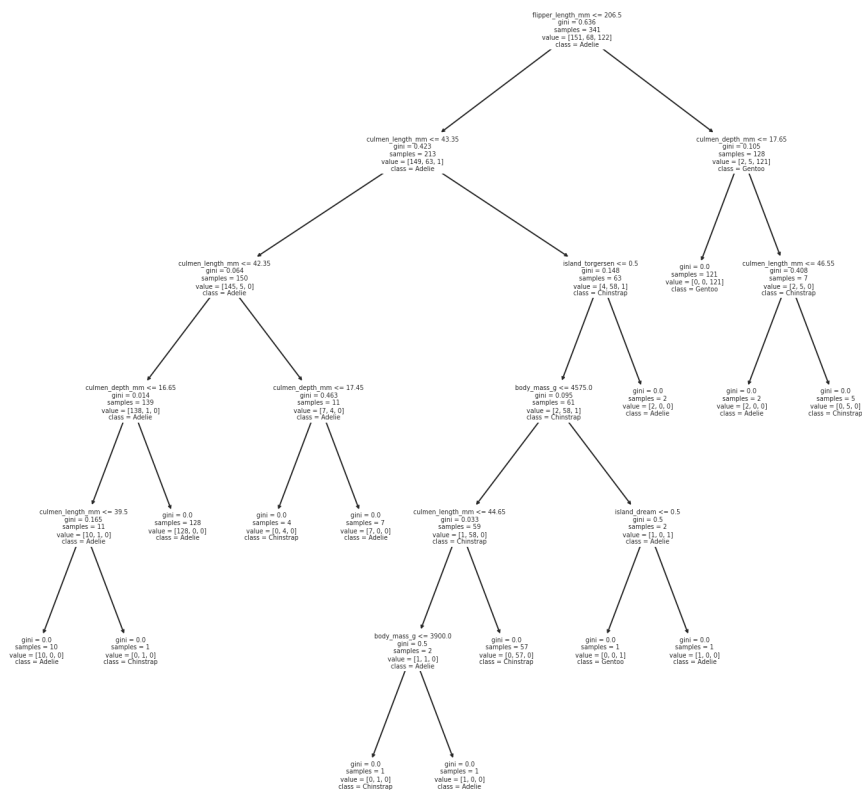
# Fitting the Decision Tree Classifier model.
DTC.fit(X, y)

# Plotting the figure with a figure size of 25 X 25.
plt.figure(figsize=(25, 25))

# Calling the plot tree with the model, feature_names, and class_names passed to it.
plot_tree(DTC, feature_names=feature_names, class_names=class_names)

# Showing the plot that was created.
plt.show()

```



```
# Importing the required libraries.
```

```
from google.colab import files
```

```
from PIL import Image
```

```
from numpy import array
```

```
# Uploading the required image.
```

```
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving scotland photo.ing to scotland photo (1).ing

```
# Opening the uploaded imag.
```

```
imgOne = Image.open('scotland_photo (1).jpg')
```

```
imgOne
```





```
# Converting the image into an array of RGB values.  
imageArray = array(imgOne)  
imageArray
```



```
# Converting the array of RGB values into a new image.  
imgTwo = Image.fromarray(imageArray.astype('uint8')).convert('RGB')  
imgTwo.save('newImage.jpg')  
imgThree = Image.open('newImage.jpg')  
imgThree
```



✓ 0s completed at 1:04 PM

